

CONTEXTGRAPH: FROM PAST REPAIRS TO EXECUTABLE REQUIREMENTS

Anonymous authors

Paper under double-blind review

ABSTRACT

A coding agent can fix the reported bug while missing a requirement already learned in an earlier repair. ContextGraph turns this experience into reusable repair memory: a requirement, the input that triggers it, and an executable observation of the desired behavior. FAISS selects candidate experiences; graph relations connect their inputs and operations so that historical conditions can be composed with the current workflow. Earlier ContextGraph retrieval experiments show the value of experience: on SWEContextBench Related-Lite, curated memory raises first-pass resolution from 14/99 to 21/99 and reaches 25 resolved tasks in two passes, which the no-memory agent reaches in three. Our executable-memory development experiments explain how history improves the repair itself. In three paired Django cases, memory adds query preservation, constraint handling, or destination-timezone behavior beyond the reported bug. A further comparison shows that composing a historical input with the current query recovers a behavior that checking the two separately misses. The central idea is to retrieve a condition the next repair should satisfy, and make that condition available to execution and refinement.

1 INTRODUCTION

Coding agents repeatedly encounter related bugs, yet each new issue describes only part of what a complete repair should accomplish. Past fixes contain knowledge that the new issue leaves implicit: which argument takes precedence, which constructor convention matters, and which neighboring operation must continue to work. Long-term memory offers a way to carry these requirements from one repair to the next.

Consider an administrative redirect that loses a deployment prefix. An agent can repair the prefix by using the full request path, yet still discard query parameters when appending a slash. A historical fix of the same API records the missing requirement: retain the query string. The history-informed agent uses `get_full_path()` with slash appending enabled, preserving both the prefix and the query. Memory contributes a behavior that the current issue did not ask the agent to rediscover.

This example suggests a useful unit of experience: a requirement together with the conditions that make it matter. A patch shows one implementation of that requirement. A sentence makes it portable. An executable witness supplies the input and observation needed to apply it accurately. ContextGraph combines these forms by storing the requirement with its triggering input and observed operation, then composing those relations with the current program.

Our investigation follows three questions: *which memory is useful, how should it be represented, and how should it be retrieved?* Historical benchmark results establish where experience helps: earlier ContextGraph systems improve single-pass resolution across several backbones and reach useful solutions with fewer retry rounds. Development interventions then identify the information responsible for a better repair. This leads to a graph whose role goes beyond ranking similar text: it supplies the relations needed to construct a relevant check from the retrieved experience.

We contribute an executable requirement representation; a retrieval interface combining FAISS candidates with graph-based input and operation composition; and empirical evidence connecting memory content to both task resolution and the behaviors preserved by the resulting repair. The resulting

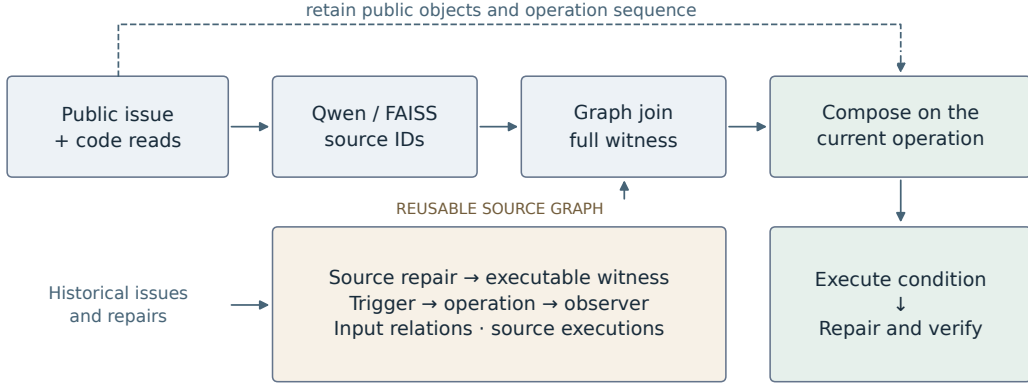


Figure 1: ContextGraph separates candidate selection from condition construction. FAISS returns source identities; the graph returns their complete witnesses and input/operation relations. Public-example composition retains the workflow while introducing a historical input condition. Execution makes its result available during repair. Source evidence is reusable; the composed check and its result belong to the current task.

design principle is simple: *remember what a repair must accomplish, and retain enough of its conditions to exercise that requirement in the next task.*

2 CONTEXTGRAPH: MEMORY AS EXECUTABLE REQUIREMENTS

2.1 REPRESENT THE BEHAVIOR LEARNED FROM A REPAIR

For a historical repair, let V^- and V^+ be the original and repaired versions. We store

$$m = (r, W, E^-, E^+), \quad W = (I, A, O), \quad (1)$$

where r states a requirement, I constructs an input, A performs the operation, and O observes whether its outcome meets r . The two executions E^- and E^+ apply the same witness to the source versions. A false-to-true change identifies a behavior learned from the repair. Neighboring inputs that pass on both versions describe behavior to preserve around that trigger.

A source writer receives the historical issue, patch, and relevant code, then constructs the requirement and witness. Separating input, operation, and observer makes the experience reusable. The agent can locate a new implementation of r while retaining the input condition that justified it. For example, named tuples and ordinary tuple subclasses can hold the same values but require different constructor calls. Their type relation is part of the requirement’s usable context.

2.2 RETRIEVE CANDIDATES, THEN RECOVER THEIR RELATIONS

The graph links a historical repair to its requirement, witness, and executions. Execution edges identify the functions reached by the witness. Input edges bind a constructor and its values to an argument of the observed operation. These relations let the retriever return a complete condition and let a composition operator determine which construction to carry into a new program.

A reader uses the public issue and original repository to identify the current operation. Qwen3-Embedding-8B/FAISS similarity selects historical sources from the same repository (Zhang et al., 2025). The current implementation keeps the top three distinct witnessed sources. Graph joins then return their requirements, complete programs, observations, and input relations. Operation connections accompany these candidates: a repair may need to preserve behavior outside the initially localized function, so an early function match does not veto a high-ranked source.

For range queries, assignment and branch analysis identifies the named-tuple constructor actually supplied to `filter(value__range=bounds)`. For timezone witnesses, the graph binds the stored instant and active timezone to the explicit `tzinfo` arguments of observed

TruncDate/TruncTime calls. The relation retains both a value and its role, allowing later checks to combine database, ambient, and destination timezones correctly.

2.3 COMPOSE HISTORY WITH THE CURRENT WORKFLOW

A historical example and a current example may each pass while their combination fails. We therefore compose the historical input condition along the current operation. Our ORM adapter lifts a scalar lookup `field=x` into a point-range lookup `field__range=R(x, x)`, using the historical two-endpoint named tuple `R`. Writing the public example as $P_q(x)$ gives

$$P_q(x) \mapsto P_q^{\text{range}}(R(x, x)). \quad (2)$$

The adapter retains the construction of x and the surrounding query. An ordinary-tuple point range supplies a neighboring check. When x is a lazy User inside an annotated subquery, retaining that object and operation sequence is essential: a lazy integer or a standalone named-tuple query exercises a different interaction.

The implementation transforms the public program’s syntax tree using the retrieved input relation. Each adapter specifies the operation it supports. The composed program retains an explicit observer: an empty fixture can test query acceptance, while a nonempty fixture tests the selected rows. This makes both preparation and semantic omissions available to the agent.

2.4 USE THE CONDITION DURING REPAIR

A complete SWE-agent receives the issue and memory, then browses, edits, runs tools, and submits a patch. Memory can supply a requirement sentence, historical implementation, or executable witness. When a witness fails on the current candidate, the interface returns its unchanged program and failed observations for refinement. A callable tool executes the supplied conditions on the working tree, keeping their inputs and observers consistent across edits.

We immediately grade submitted repairs with the benchmark’s official verifier and execute historical, current, and joint requirements on the repair. Joint checks measure the additional behavior supplied by memory; neighboring checks measure the behavior preserved while adding it. Fixture failures are recorded separately from observed violations of the checked requirement. This interface connects three stages: selecting an experience, exercising its condition, and incorporating that behavior into the submitted implementation.

3 EXPERIENCE IMPROVES RESOLUTION AND REDUCES RETRY ROUNDS

Experimental setting. We report two stages of ContextGraph. The historical retrieval systems store rules or repair episodes and expose them through a memory tool. Their results on SWE-bench Verified and SWEContextBench (Jimenez et al., 2024; Zhu et al., 2026) are restored from the project’s recorded experiment tables. The executable requirement method in Section 2 is evaluated through the development interventions in Sections 4–6. The historical tables retain their original task sets, backbones, and attempt regimes; they measure those earlier systems. Appendix A collects the remaining tables and protocol details.

Useful experience transfers across backbones. Table 1 compares six memory conditions on the historical Related-Lite98 single-shot protocol. ContextGraph has the highest resolution rate among the learned retrieval conditions on all four backbones. With GPT-5.4, it resolves 34/98 tasks against 21/98 without memory; with Kimi K2.6, it resolves 28/98 against 18/97. The oracle-summary condition supplies the benchmark’s paired experience directly. ContextGraph’s 34/98 on GPT-5.4 is close to its 35/98, showing the value of finding useful experience without being given that pairing at query time.

Memory gets to useful solutions earlier. A separate Related-Lite99 campaign uses DeepSeek V4 Pro with a 500-call and \$30 per-task budget in both arms. Its curated episodic memory retrieves reference fixes, verification hints, and pre-base repository history at two moments: before choosing an approach and after an error. The served pool contains 291 static episodes. Each later pass retries

Table 1: Historical Related-Lite98, one attempt per task (V_1). Cells show resolved/completed; bold marks the best non-oracle condition by rate. All methods use the same 300-summary pool within each backbone.

Memory	GPT-5.4	DeepSeek V4 Pro	Kimi K2.6	MiniMax M2.7
None	21/98	30/98	18/97	19/98
Random summaries	19/97	26/96	25/98	19/97
Mem0	19/98	26/97	25/98	18/98
LangMem	24/98	33/97	24/98	21/98
Oracle summary	35/98	27/97	24/98	15/95
ContextGraph	34/98	34/97	28/98	27/98

Table 2: Historical Related-Lite99 with curated episodic memory. Resolved counts are cumulative. Wins/losses compare ContextGraph with control; p is the paired exact McNemar test.

Retry budget	No memory	ContextGraph	Wins/losses	p
1 pass	14/99	21/99	7/0	.016
2 passes	16/99	25/99	9/0	.0039
3 passes	25/99	27/99	6/4	.75

only that arm’s unresolved tasks. Table 2 shows the practical benefit: memory reaches 25 resolved tasks after two passes, while the control reaches 25 after three. The first two passes add seven and nine resolutions respectively, with no control-only successes. By the third pass the gap narrows to two tasks. This pattern locates the strongest observed advantage in earlier successful repair. Excluding ten targets with near-duplicate source fixes retains a first-pass gain of 19/89 versus 13/89 ($p = .031$). A separate replication across three additional backbones also raises first-pass resolution (Appendix Table 8).

Available experience matters. The pool-size experiment holds the historical GPT-5.4 agent and retrieval procedure fixed while sampling 50, 100, 200, or 300 summaries (Table 3). The full pool gives the best observed result, 34/98; the smaller pools yield 21–27 resolutions. The pattern motivates a coverage-first construction strategy: establish that useful source experience exists before optimizing its ranking. Our executable memory applies this principle at the level of a supported behavioral condition.

From retrieval gains to a better memory unit. On the historical full Verified500 evaluation, all three memory methods improve the point estimate over no memory, with FAISS leading this comparison (Table 4). These outcomes make FAISS a natural component of ContextGraph: dense similarity efficiently proposes useful sources. The next question is what the retrieved source contributes to the actual repair. Our development experiments address that question by identifying requirements that memory adds, then representing the relations needed to apply them.

4 HISTORY SUPPLIES A MISSING REPAIR REQUIREMENT

The executable-memory experiments use a Verified split of 280 source tasks, 70 development tasks, and 150 held-out evaluation tasks. The cases below are development interventions with DeepSeek V4 Pro through OpenRouter, temperature zero and matched budgets within each comparison. They measure whether history adds a required behavior to the submitted repair. Table 5 presents three such additions. Django14404 uses the first-ranked source issue and patch; Django12708 and Django11138 come from a fixed six-task cohort receiving three unchanged Flat rules per task. Both arms pass official verification on all six cohort tasks. The historical requirements distinguish the completeness of those repairs.

An omitted behavior of the same API. Django14404 reports that administrative redirects lose a deployment prefix. Both agents fix the prefix and pass official verification, but only the re-

Table 3: Historical Related-Lite98 memory-pool ablation, GPT-5.4, single-shot V_1 . The separate best-of-two V_2 result is in Appendix Table 9.

Number of summaries	50	100	200	300
Resolved/completed	21/96	27/98	26/97	34/98
Resolution rate	21.88%	27.55%	26.80%	34.69%

Table 4: Historical SWE-bench Verified500, GPT-5.4, 75 steps and uncapped per-task cost. Rates use all 500 tasks, including unfinished attempts. Wins/losses and exact McNemar p compare each method with no memory.

Method	Resolved	Rate	Gain (pp)	Wins/losses	p
No memory	309	61.80%	—	—	—
ExpeL	312	62.40%	+0.6	43/40	.826
FAISS	327	65.40%	+3.6	49/31	.057
ContextGraph	318	63.60%	+1.8	50/41	.402

pair informed by source16612 also preserves the query string. A joint input combines the prefix with repeated and percent-encoded query values, `keep=a%2Fb&keep=c%20d`. Only the history-informed repair preserves both.

A neighboring operation. Django12708 reports ambiguous deletion of an ordinary index when a unique constraint uses the same columns. The first retrieved source, 15732, instead concerns deletion of a unique constraint sharing its column with a primary key. Both repairs fix ordinary-index selection; only the memory-informed repair also excludes primary keys when deleting unique constraints. The source-generated PostgreSQL check runs unchanged on both target repairs. A separate combined input creates an ordinary index, redundant uniqueness, and a primary key on the same column. Both repairs preserve ordinary-index deletion, but only the memory-informed repair also deletes redundant uniqueness while retaining the primary key and data. History supplies a requirement for the neighboring operation.

An independent argument of the same operation. Django11138 requires date conversions to respect the database’s source timezone. Source13363 requires `TruncDate` and `TruncTime` to honor an expression’s explicit destination timezone. Both agents repair the source timezone, but only the memory-informed repair also honors the destination. A source-generated check distinguishes the historical versions and runs unchanged on the target repairs. On an independent SQLite input combining a UTC+1 database with an explicit New York destination, only the memory-informed repair returns the correct date and time. The issue and history constrain different ends of the conversion.

The addition can guide further repair. Two fresh agents start from the same completed ordinary timezone patch. Issue-based additional review returns it unchanged; supplying a generated historical check and its failed observations leads the other agent to use `self.get_tzname()` in both truncation methods. Both revisions pass official verification, but only the latter passes the historical and joint inputs. The intervention supplies corrective information that further issue-based review does not recover in this comparison.

5 WHAT INFORMATION MAKES MEMORY USABLE?

A requirement can suffice without the source implementation. On the redirect task, replacing the first entry of the existing Flat packet with its original issue and patch yields a byte-identical repair; both forms preserve the joint behavior. Two new agents then revise the completed ordinary patch. Both receive the source-generated sentence, “The redirect location from Admin-Site.catch_all_view includes the original query string.” One also receives the generated check and its failed observations. Both submit byte-identical repairs that pass official and joint checks. The sentence-only agent finds `get_full_path()` in the current repository: memory communicates

Table 5: Joint current/historical behavior in complete-agent comparisons. Each displayed repair passes official verification. The columns compare the actual repairs with and without memory; the rows retain their distinct task requirements.

Target	Historical addition	No memory	Memory
Django14404	Preserve query string	Fail	Pass
Django12708	Delete redundant uniqueness	Fail	Pass
Django11138	Honor destination timezone	Fail	Pass

Table 6: Memory-content interventions: one fresh attempt per input and development task. All eight pass official verification. MySQL and named-zone checks were constructed after inspecting the timezone repairs and applied to all four submissions.

Memory input	Constraint joint	Timezone joint	MySQL cases	Named-zone cases
None	Fail	Fail	Pass	Fail
Requirement	Fail	Pass	Pass	Pass
Implementation	Pass	Pass	Pass	Fail
Both	Pass	Pass	Fail	Pass

the desired behavior while the agent locates its implementation. Here the portable requirement is enough for the agent to find and use the right implementation.

Which information makes the requirement usable? We select the constraint and timezone benefit cases and run four fresh complete agents per task, fixing the single historical source and its delivery before editing. Two source-only constructions produce a behavioral requirement and an implementation reference; the combined input concatenates them unchanged. All eight repairs pass official verification (Table 6). On Django12708, only Implementation and Both also remove redundant uniqueness while preserving the primary key. The Requirement agent tests a field with `unique=True`, but places `unique_together` on a different, two-column set. This nearby passing test loses the same-column condition stated in memory. On Django11138, all three memory inputs preserve the explicit destination timezone, while the fresh no-memory repair omits it. The Requirement agent finds `self.get_tzname()` in the target implementation without being given that helper in memory.

The timezone repairs also differ on further conditions. Both still generates an unnecessary MySQL conversion between identical zones; Implementation instead misinterprets a named Paris source timezone, producing New York time 19:21 instead of 17:30 on a summer input. Requirement passes both additional diagnostics. These concrete omissions distinguish carrying a requirement, applying its triggering condition, and implementing the resulting behavior correctly. Two subsequent original-base agents receive the unchanged constraint Requirement, with or without a source-validated example; both pass official and joint checks. The same requirement succeeds when its trigger is reconstructed correctly; preserving that input construction is a useful role for the graph.

Execution feedback makes an omitted condition concrete. Two fresh agents refine the same original failed Requirement candidate, both receiving its requirement again. One also receives the unchanged source-generated check and its actual failures for a field with a primary key or `unique=True`. Requirement-only review excludes the primary key but still fails the ordinary unique-field condition. Feedback adds generated-name disambiguation and passes both conditions. Both revisions pass official verification and the withheld joint sequence. The added behavior is recovery of the ordinary unique-field condition identified by feedback.

6 GRAPH COMPOSITION MAKES EXPERIENCE ACTIONABLE

Preserve the input relation, not just its values. On Django12663, both a three-source code-connected packet and a ten-source similarity packet contain the historical named-tuple repair, source13590, yet neither initial repair recovers its behavior. Three fresh revisions start from the

	Official verification	Historical named tuples (2)	Named tuple + lazy User (1)	ADDITIONAL DIAGNOSTIC Ordinary tuple/list subclasses (2)
Memory supplied for revision				
Original issue + patch	Pass	Pass	Pass	Pass
Requirement sentence	Pass	Fail	Fail	Pass
Requirement + check + execution failures	Pass	Pass	Pass	Fail

Figure 2: Memory content changes which behaviors a repair preserves. One revision per input starts from the same Django12663 candidate and source13590. Parentheses give case counts; each cell reports all cases in its group. The ordinary-subclass inputs were chosen after inspecting the feedback repair. All three repairs pass official verification, while only original history passes every displayed behavior.

same candidate and source (Figure 2). Original history and execution feedback recover named tuples; the requirement sentence alone does not. The feedback repair expands every non-builtin sequence into positional arguments, which also breaks ordinary tuple and list subclasses. Original history retains the constructor distinction and passes all displayed behaviors. This is precisely the relation the graph should carry: the argument convention of the input’s type, alongside its endpoint values.

Join the historical input to the current operation. The current issue involves a lazy User in a nested query; history contributes a named-tuple range. On an earlier partial repair, the current scalar query and the historical range query each pass, while a query combining them fails. The composed condition reveals an omission in the second container reconstruction. Four fresh refinements of that candidate illustrate its use: history alone yields one of two repairs passing all seven measured conditions; adding the constructed joint program and its failed observation yields two of two. All four pass official verification. Both augmented agents execute the joint input and repair the second reconstruction.

The deterministic adapter in Equation 2 reaches the same omission while retaining the public program’s lazy User and nested annotation. It produces a named-tuple point range and an ordinary-tuple neighbor. This is the graph’s constructive contribution: an input relation from a past repair becomes a concrete check along the current query.

A matched comparison isolates the added condition. A new original-base pair receives identical history, public scalar program, its observed failure, a callable execution tool, and instructions to run the supplied conditions before editing and submission. The augmented arm also receives the two composed programs and their failures. Both repairs pass official verification (Table 7). The current-feedback repair fails both historical examples and the withheld named-tuple/lazy-User query; the augmented repair passes those behaviors and preserves the ordinary subclasses. The augmented agent runs the joint input before editing and again after removing a global conversion that broke an existing test. Its remaining range-endpoint failure directs the final correction.

Retain candidates that can supply an omitted condition. In this same case, demanding an operation match from the initial public-code localization excludes source13590, which FAISS ranks first. Keeping the similarity candidate lets the graph return its input relation and construct the joint program above. This motivates the division in Figure 1: FAISS proposes sources; graph relations support their adaptation to the current operation. The usefulness of a candidate is established by what its condition adds to the repair.

Compose roles to expose a broader requirement. The timezone relation provides another example. A fixed adapter varies the historical witness’s three zones across database, ambient, and destination roles and shifts the input by 182 days. Of 72 combinations, the historical patch im-

Table 7: Added conditions in the original-base Django12663 pair. Both inputs contain the same history, current program and failure, tool, and execution instruction. Only the second adds the composed programs and their failures. Each row counts the named checks, not independent tasks.

Measured behavior	Current feedback	+ Composed conditions
Official target verification	Pass	Pass
Historical named-tuple examples	0/2	2/2
Withheld nonempty joint queries	2/3	3/3
Withheld ordinary subclasses	2/2	2/2
Mechanical scalar / named tuple / tuple	2/3	3/3

proves 20, preserves 20, and leaves 32 incorrect. Applying the same family to the four timezone content interventions shows that Requirement and Both pass all 72. Implementation passes the 40 source-supported combinations but fails the remaining 32, including 16 passed without memory. Role composition exposes how the current source-timezone requirement interacts with the historical destination requirement. A complete repair can therefore satisfy combinations beyond those corrected by the historical implementation.

Design implication. A useful memory communicates three things together: the behavior to achieve, the input relation under which it matters, and an observation that can guide an edit. A concise sentence sometimes supplies all the agent needs. When adaptation loses a condition, the graph reconstructs it as an executable input; feedback then turns the remaining violation into a concrete repair objective. This progression explains why storing more related text alone can leave the same bug unresolved, and why preserving a small but decisive relation can improve the resulting implementation.

7 RELATED WORK

Graph memory supports associative retrieval through links between experiences and concepts, as in HippoRAG (Gutiérrez et al., 2024); ExpeL extracts reusable insights from success and failure (Zhao et al., 2024). MemOp optimizes memory writing from downstream success, localization, and efficiency (Guo et al., 2026), while ExpGraph learns retrieval from task scores with and without experience (Feng et al., 2026). Our requirement-level interventions identify what useful memory contributes to the implementation, including behaviors that differ between repairs with the same official target score.

ExpeRepair generates reproduction and boundary tests from historical demonstrations (Mu et al., 2026); RETester transforms historical bug templates while preserving the relevant operation (Wang et al., 2025). MIRA separates SQL corrections into reusable requirements and uses execution to guide adaptation (Liu et al., 2026). TRACE compiles user corrections into rules and runtime checks (Zhou et al., 2026). ContextGraph connects the historical input relation to the current operation, allowing a joint condition to expose an omission that replaying either example separately misses.

8 CONCLUSION

ContextGraph treats past repairs as reusable requirements for future work. Historical retrieval results show that experience can improve first-pass resolution and reduce the retry rounds needed to reach useful solutions. The executable-memory experiments identify a complementary advantage: recovering omitted behaviors and preserving them through adaptation. Combining FAISS candidate selection with graph-based condition construction provides a concrete path from a relevant past repair to a better current one.

AI USE STATEMENT

Generative AI assisted literature review, method development, implementation, experiments, analysis, and writing.

REFERENCES

- Tao Feng, Chongrui Ye, Tianyang Luo, Jingjun Xu, Xueqiang Xu, Haozhen Zhang, Zhigang Hua, Yan Xie, Shuang Yang, Ge Liu, and Jiaxuan You. ExpGraph: Model-agnostic experience learning with graph-structured memory for LLM agents, 2026. URL <https://arxiv.org/abs/2605.30712v1>.
- Xuehang Guo, Zora Zhiruo Wang, Qingyun Wang, Graham Neubig, and Xingyao Wang. Enhancing software engineering through closed-loop memory optimization, 2026. URL <https://arxiv.org/abs/2606.05646v1>.
- Bernal Jiménez Gutiérrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models, 2024. URL <https://arxiv.org/abs/2405.14831v3>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues?, 2024. URL <https://arxiv.org/abs/2310.06770>.
- Yining Liu, Chenyu Yang, Boyan Li, Rui Mao, and Yuyu Luo. MIRA: Evidence-verified repair memory for text-to-sql correction, 2026. URL <https://arxiv.org/abs/2608.06950v1>.
- Fangwen Mu, Junjie Wang, Lin Shi, Song Wang, Shoubin Li, and Qing Wang. ExpeRepair: Dual-memory enhanced LLM-based repository-level program repair, 2026. URL <https://arxiv.org/abs/2506.10484v2>. Version 2; accepted at FSE 2026.
- Haibo Wang, Zhuolin Xu, and Shin Hwei Tan. Testing refactoring engine via historical bug report driven llm, 2025. URL <https://arxiv.org/abs/2501.09879v2>.
- Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models, 2025. URL <https://arxiv.org/abs/2506.05176v3>.
- Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. Expel: Llm agents are experiential learners, 2024. URL <https://arxiv.org/abs/2308.10144v3>.
- Yujun Zhou, Kehan Guo, Haomin Zhuang, Xiangqi Wang, Yue Huang, Zhenwen Liang, Pin-Yu Chen, Tian Gao, Nuno Moniz, Nitesh V. Chawla, and Xiangliang Zhang. Getting better at working with you: Compiling user corrections into runtime enforcement for coding agents, 2026. URL <https://arxiv.org/abs/2606.13174v1>.
- Jiayuan Zhu, Junde Wu, Minhao Hu, Shengda Zhu, Jiazhen Pan, Weixiang Shen, Yijun Yang, Fenglin Liu, Jianye Hao, Yueming Jin, Qirong Ho, and Min Xu. Swe context bench: A benchmark for context learning in coding, 2026. URL <https://arxiv.org/abs/2602.08316v3>. Version 3.

A HISTORICAL BENCHMARK TABLES AND PROTOCOLS

The benchmark tables recover the project’s historical aggregate records from `main.tex` (paper revision 3e51135) and earlier ICLR drafts. This revision restores the reported data; it adds no solver execution or new verification of those historical patches. Verbatim source tables and a source map accompany the paper.

Memory configurations. The Verified500 system indexes trajectory fragments, rules, and communities with dense, lexical, and graph retrieval. The Related-Lite98 system indexes 300 Experience summaries with repository-filtered lookup and a memory tool; its production repository-filtered route differs from generic channel fusion. The Related-Lite99 episodic system serves reference fixes, verification hints, and pre-base history from 291 episodes at pre-plan and post-error query points. These configurations precede the Qwen executable-witness graph.

Task counts and attempts. Verified500 uses a fixed 500-task denominator; completed counts were 486 for no memory, 488 for ExpeL, 490 for FAISS, and 484 for ContextGraph. Related-Lite98 excludes one shared image-build failure from the original 99-task manifest and reports resolved/completed in each arm. Its V_1 regime uses one attempt; the historical V_2 aggregate includes failure reruns. The main-text pool-size table uses V_1 only: the old delta column used the V_2 control rate and is omitted. The MiniMax M2.7 campaign used a gateway-compatibility shim.

Related-Lite99 source content. The episodic campaign served static source episodes: its original online deposit path omitted fields needed for retrieval. The historical overlap analysis identified ten targets with near-duplicate source fixes (token-Jaccard at least 0.85); removing them gives an 89-task frame. DeepSeek V4 Pro retains a first-pass gain of 19/89 versus 13/89 ($p = .031$). The full-set cumulative two-pass gain of nine exceeds the campaign’s preregistered threshold of eight.

Table 8: Historical curated-episode Related-Lite99 replication, one pass per backbone. Last column gives recorded control/memory counts and paired p after removing ten near-duplicate targets; arm-specific completed denominators for that subset are not provided in the original table.

Backbone	Control	Memory	C/M only	p	Reduced set
DeepSeek V4 Pro	14/99	21/99	0/7	.016	13/19 ($p = .031$)
DeepSeek V3.2 exp	16/98	20/99	4/8	.39	15/18 ($p = .55$)
GLM 5.2	17/99	24/99	1/8	.039	15/21 ($p = .070$)
Kimi K3	29/99	31/99	3/5	.73	26/28 ($p = .73$)

Table 9: Historical Related-Lite98 GPT-5.4 aggregate with failure reruns (V_2 reporting regime). The historical table alone does not supply uniform per-arm retry accounting; Table 1 gives the separate single-shot V_1 comparison.

Memory	Resolved/completed	Rate
Random summaries	19/97	19.59%
Mem0	19/98	19.39%
LangMem	24/98	24.49%
No memory	25/98	25.51%
Oracle summary	35/98	35.71%
ContextGraph	41/98	41.84%

Table 10: Historical Verified development subset. Rates use completed evaluations. The preprint transcribed this table from an experiment screenshot; 66.6% retains the recorded display precision.

Method	Completed/planned	Resolved	Recorded rate
ContextGraph	48/50	32	66.6%
ExpeL	50/50	32	64.0%
FAISS	49/50	30	61.2%
No memory	50/50	29	58.0%

Table 11: Historical SWE-bench-Pro50, recovered from a disabled preprint section. Memory arms share a 3,183-strategy pool and a \$10 per-task cap; no memory is uncapped. Rates use completed evaluations; costs are recorded totals under these different budgets.

Method	Completed/planned	Resolved	Rate	Cost (\$)
ContextGraph	47/50	11	23.40%	41
FAISS	47/50	5	10.64%	68
No memory	46/50	2	4.35%	1,686

Table 12: Historical trajectory-graph snapshot, using text-embedding-3-large vectors with 3,072 dimensions. The new executable-witness graph uses a separate Qwen embedding space.

Node label	Count	Role
Trajectory	1,795	Source episode
Fragment	13,813	Action and observation slice
ErrorPattern	586	Error type and keywords
Strategy	9,600	Extracted rule
CanonicalRule	7,835	Consolidated rule
PlaybookEntry	8,910	Retrieval-ready entry
Community	781	Linked-fragment summary
ProblemSummary	1,795	Episode problem abstraction
Total nodes	45,115	
Total edges	73,975	Seven typed relations

B EARLIER INTERFACES AND DEVELOPMENT COMPARISONS

Table 13 restores the earlier ICLR draft’s six-interface Related-Lite99 experiment: a separate DeepSeek V4 Pro campaign with 500 calls and the last five observations retained. Its frozen verdict requires fail-to-pass success and does not veto every pass-to-pass regression. The 88-task frame belongs to this campaign’s source-overlap analysis.

Table 13: Earlier Related-Lite99 memory interfaces and their original 88-task sensitivity frame. Gains and paired exact tests use the full 99-task frame.

Memory	Resolved/99	Resolved/88	Gain (pp)	<i>p</i>
No memory	17	16	—	—
Flat patches	18	18	+1.01	1.000
Patches + procedure card	18	15	+1.01	1.000
Graph	12	12	-5.05	.227
Rules	13	12	-4.04	.424
File list	9	8	-8.08	.057

Table 14: Restored Django14404 development comparison. All candidates pass the official 2 fail-to-pass and 323 pass-to-pass tests.

Candidate	Prefix	Query	Prefix + query
Initial repair	Pass	Fail	Fail
Issue-based revision	Pass	Fail	Fail
Historical patch revision	Pass	Pass	Pass
Executed relation revision	Pass	Pass	Pass

Intervention scope. Content comparisons start fresh agents from the original development target; refinement comparisons start from a common completed candidate. Additional MySQL, named-zone, and ordinary-subclass checks were constructed after inspecting the corresponding repairs, then applied across their comparison group. The composition pair holds history, current example and failure, and the execution interface fixed; the added input is the composed condition.

Operation-dependent preservation. In Matplotlib20826, clearing and replotting twin axes restores finite data bounds, so failure of an isolated historical plotting example does not describe the state after that workflow. In Django16819, canceling a same-name index remove/add pair loses a required change of indexed column. These observations motivate applying memory conditions along the current operation sequence.

Four-benchmark extension. The new matched no-memory, FAISS, and Graph evaluation targets Verified150, Related-Lite99, LoLBench/CAPBench100, and Datacurve DeepSWE113. These four new-method result sets are incomplete. The closed DeepSWE Kea development baseline passes 9/12 fail-to-pass and 139/139 pass-to-pass tests (binary reward 0). The closed JDK5 LoLBench baseline originally submitted no artifact; a separate no-inference recovery of its recorded patch passes 0/5 fail-to-pass and 51/51 pass-to-pass tests (binary reward 0). These are development outcomes. The historical Verified500 table is a different evaluation from Verified150.

C PLANNED EXPERIMENTS: RESOLUTION AND EFFICIENCY

These tables specify the experiments needed to extend the paper’s argument. *TBD* denotes an unmeasured result, not a projected score. Sample sizes are proposed allocations. All new solver experiments use the authorized DeepSeek V4 Pro OpenRouter route; new embeddings use the pinned Qwen3-Embedding-8B space. Each submission is verified immediately with its benchmark’s verifier; independent semantic trajectory reviews occur at most ten minutes apart.

Table 15: **Does the complete method improve repair success?** Planned matched comparison on the four fixed manifests. Each cell will report resolved/planned and assessed/planned for one full first pass. Every arm uses the same agent, model, tools, per-task budget, and task order. Flat and Graph use the same frozen source bank. Flat returns complete memory records by FAISS similarity; Graph adds the method’s relations and condition composition. Graph-minus-Flat is the primary method comparison; the final row reports its paired task difference and 95% interval separately for each benchmark.

Method	Verified	Related-Lite	LoLBench	DeepSWE
Planned tasks	150	99	100	113
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Graph — Flat (pp, CI)	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 16: **Does memory reach useful solutions with less computation?** Extend the same frozen comparison to three real passes, retrying each arm’s own unresolved tasks. Produce this panel separately for each benchmark. Report cumulative resolved/planned, all-attempt tokens and USD per resolved task after three passes, and aggregate active seconds to resolve 10% of the fixed manifest. Active time includes model, tool, and verification execution; worker allocation and task order are matched. A threshold never reached is reported as such. Construction cost is added through Table 24. The comparison tests the early-resolution pattern in Table 2 using measured time and cost.

Method	Pass@1	Pass@2	Pass@3	Tokens/solve	USD/solve	Time to 10%
No memory	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 17: **Order the work around the scientific argument.** The experiment sequence uses development tasks to determine useful content and fix the method before the four-manifest comparison. Each later stage answers a distinct question; numerical gains are measured rather than assumed.

Stage	Question	Experiment and evidence
1	What should memory communicate?	Content and source-utility interventions in Tables 18–19.
2	What must the graph preserve?	Relation and composition ablations in Table 20.
3	Can retrieval find useful conditions?	Ranking and source coverage in Tables 21–23.
4	Does the fixed method help in practice?	Four-benchmark resolution, retry efficiency, and amortized cost in Tables 15, 16, and 24.

D PLANNED EXPERIMENTS: WHICH MEMORY IS EFFECTIVE?

Use 24 distinct development targets from the existing dev70 partition, spanning at least four repositories and four operation families. Select task/source pairs from public issues and source evidence before observing the new repairs. Use three independent repetitions per arm, with common target and source IDs. The proposed allocation is 72 attempts per arm; repetitions remain clustered by target in effect estimates. The examples motivating Tables 6 and 7 are a development starting point for this expansion.

Table 18: **Which representation transfers the missing requirement?** Each target uses the same single historical source and the same solve budget. The rows vary only the delivered content: requirement r , historical patch H , witness $W = (I, A, O)$, and source executions E^- , E^+ . All rows receive the same execution tool and current-task instructions. Official is benchmark resolution; Joint requires all predeclared combined current/historical checks; Preserve requires the neighboring input checks; Complete requires all three on the same submitted repair. Additional joint and neighboring evaluation inputs and observers are fixed before these runs and withheld from the solver; delivered source witnesses remain visible. Report mean task success across repetitions and paired intervals.

Delivered memory	Official $\uparrow$	Joint $\uparrow$	Preserve $\uparrow$	Complete $\uparrow$
None	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Requirement r	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Historical implementation H	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
$r + H$	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
$r + W$	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
$r + W + E^- + E^+$	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 19: **What makes one source experience useful?** On a 12-target subset chosen before candidate outcomes, form a pool of four distinct sources per target using dense, lexical, operation-connected, and same-repository random selection, filling duplicates from the same policy. Deliver each source individually in the complete-witness format for three repetitions: 144 source-conditioned attempts, plus the matched no-memory controls from Table 18. Classify source/current relations from their original programs before inspecting solver repairs. Report the source-pair count and the paired change from no memory in Complete success and preservation failures. All sources share one target-level evaluator containing the applicable current and historical requirements. These per-source effects supply development utility labels for Table 21.

Source/current relation	Source pairs	Δ Complete	Δ regressions
Adds an omitted requirement	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Preserves already-satisfied behavior	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Shares operation, changes input relation	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Different operation or conflicting requirement	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

E PLANNED EXPERIMENTS: GRAPH RELATIONS AND RETRIEVAL

Table 20: **Which graph operation improves the submitted repair?** Use the same 24 development targets and three repetitions, fixing the FAISS top-three source IDs, complete source content, memory token allowance, current feedback, and callable tool. A delivers the source witnesses. B asks a writer to generate joint checks from the same text records and public example; C composes checks using the graph’s explicit input and operation bindings. Match the total generation-plus-solver budget. D and E remove one relation family from C while retaining nodes, text, and source IDs; a missing binding suppresses its composed check while leaving the source witness available. F adds failed execution observations to C. Compare B–A for generated joint checks, C–B for graph-bound construction, C–D/E for relation types, and F–C for execution feedback. All arms have the same tool budget. Alongside Complete success, count repair incorporation only when the submitted implementation passes the relevant joint checks.

Variant	Joint $\uparrow$	Complete $\uparrow$	Regressions $\downarrow$
A. Flat complete witnesses; no composition	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
B. A + text-based joint-check generation	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
C. A + graph-bound condition composition	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
D. C without input-role relations	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
E. C without operation/argument bindings	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
F. C + executed-condition feedback	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 21: **Can a retriever find behaviorally useful memory?** Panel A ranks the four-source pools from Table 19; panel B ranks the entire frozen source bank on all 24 development targets. For B, measure standalone source utility on the union of all policies’ top-three sources, using the same intervention as Table 19. Hold the complete-witness payload and $k = 3$ fixed. Utility@3 is the mean measured standalone Δ Complete of the selected sources; report it with uncertainty across targets. A full graph policy is then fixed on development data for Table 15. As an end-to-end development check, run each top-three packet with three repetitions and the same composer; the Complete column measures the packet’s actual combined effect. Composable@3 is the fraction of targets with at least one retrieved condition that composes with the current workflow and executes with its observer. The operation-reranking row is a proposed policy; the current implementation retains FAISS order and supplies operation links without a hard veto.

Candidate ranking	Utility@3	Composable@3	Complete	Retrieval ms
A. Controlled four-source pools; 12 development targets				
FAISS dense similarity	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
BM25	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + BM25 reciprocal-rank fusion	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + soft operation reranking	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense with a required operation match	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
B. Full source bank; 24 development targets				
FAISS dense similarity	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
BM25	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + BM25 reciprocal-rank fusion	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense + soft operation reranking	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Dense with a required operation match	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

F PLANNED EXPERIMENTS: COVERAGE, SCALE, AND REUSE

Table 22: **Can the bank supply usable experience across the four benchmarks?** The source273 column is an existing measured same-repository witness-coverage count. Expand sources using official training data and commits preceding each pinned target base; construct source witnesses before freezing the bank shared by Flat and Graph. Planned columns measure source-level false-to-true witnesses, coverage after expansion, and the fraction of attempted source/current compositions that execute normally with their observer. Coverage is source availability, while repair benefit is measured in Table 15. This experiment develops the source bank needed for LoLBench and DeepSWE.

Benchmark	Source273 coverage	New witnesses	Expanded coverage	Executable joins
Verified150	133/150	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Related-Lite99	91/99	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
LoLBench100	2/100	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
DeepSWE113	0/113	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>

Table 23: **Does scale help through useful-source coverage or ranking?** Use the 12-target source-utility study with a fixed delivered $k = 3$. First add 0, 4, or 16 same-repository distractors per target while retaining its original four-source pool; measure selection stability and Complete success for FAISS and the chosen graph policy. Then replace the source with the highest measured standalone utility with an additional distractor, keeping the pool size fixed. Freeze these manipulations before new packet outcomes and run three repetitions per cell. Distractors have complete witnesses for other operations, selected from public source bindings. This development experiment uses measured utility labels and precedes the fixed-manifest comparison.

Pool condition	Size	Best source in top3	Flat Complete	Graph Complete
Original candidate pool	4	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Add 4 distractors	8	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Add 16 distractors	20	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
Replace best source, fixed size	20	n/a	<i>TBD</i>	<i>TBD</i>

Table 24: **When does executable memory repay its construction cost?** Measure actual source construction, before/after execution, embedding, and indexing charges separately from solve-time retrieval and condition execution. For each benchmark and method, report the amortized cost $[C_{\text{build}} + C_{\text{solve}}(N)]/R(N)$ at the listed cumulative task counts, using one fixed task order. $R(N)$ counts official resolutions and solve cost includes failed attempts. Prefixes beyond a benchmark’s manifest, or with $R(N) = 0$, are n/a. For each memory method, report the first observed prefix with at least the control’s resolutions and lower total cost per resolution, or “not reached.” Use measured costs at these prefixes rather than projecting a success rate.

Method	Build USD	USD/solve at 25	At 50	At 100	Break-even tasks
No memory	n/a	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	Reference
FAISS Flat	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>
ContextGraph	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>	<i>TBD</i>